局部类型参数合成
(Local Type Argument Synthesis)

A Very Informal Tutorial

Koshiba / 2021.07

关于类型参数

关于类型参数

- 什么是<u>类型参数</u>（*type arguments*）

关于类型参数

- 什么是类型参数（*type arguments*）
  - 参数化多态（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables

关于类型参数

- 什么是类型参数（*type arguments*）
  - 参数化多态（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables
  - 既有 term-level variables

关于类型参数

- 什么是类型参数（*type arguments*）
  - 参数化多态（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables
  - 既有 term-level variables
  - 又有 type-level variables

关于类型参数

- 什么是<u>类型参数</u>（*type arguments*）
  - <u>参数化多态</u>（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables
  - 既有 term-level variables
  - 又有 type-level variables
- 如给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$，则 f

关于类型参数

- 什么是类型参数（*type arguments*）
  - 参数化多态（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables
  - 既有 term-level variables
  - 又有 type-level variables
- 如给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$，则 f
  - 有 2 个 type-level variables T_1, T_2，1 个 $T_1 \to T_2$ 类型的和 1 个 $List\langle T_1 \rangle$ 类型的 term-level variables

关于类型参数

- 什么是类型参数（*type arguments*）
  - 参数化多态（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables
  - 既有 term-level variables
  - 又有 type-level variables
- 如给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$，则 f
  - 有 2 个 type-level variables T_1, T_2，1 个 $T_1 \to T_2$ 类型的和 1 个 $List\langle T_1 \rangle$ 类型的 term-level variables
- f 是多态的，因此它的调用

关于类型参数

- 什么是类型参数（*type arguments*）
  - 参数化多态（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables
  - 既有 term-level variables
  - 又有 type-level variables
- 如给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$，则 f
  - 有 2 个 type-level variables T_1, T_2，1 个 $T_1 \to T_2$ 类型的和 1 个 $List\langle T_1 \rangle$ 类型的 term-level variables
- f 是多态的，因此它的调用
  - 可以是 $f\langle Char, Bool \rangle(\lambda x : Char \to true, ['a', 'b', 'c'])$，也可以是 $f\langle Int, Int \rangle(\lambda x : Int \to x + 1, [1,2,3])$

关于类型参数

- 什么是类型参数（*type arguments*）
  - 参数化多态（*parametric polymorphism*）函数的 type-level variables，简称 **ty**pe **var**iables
  - 既有 term-level variables
  - 又有 type-level variables
- 如给定 $f : \forall T_1, T_2 .\ ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$，则 f
  - 有 2 个 type-level variables T_1, T_2，1 个 $T_1 \to T_2$ 类型的和 1 个 $List\langle T_1 \rangle$ 类型的 term-level variables
- f 是多态的，因此它的调用
  - 可以是 $f\langle Char, Bool \rangle(\lambda x : Char \to true, ['a', 'b', 'c'])$，也可以是 $f\langle Int, Int \rangle(\lambda x : Int \to x + 1, [1,2,3])$
  - 例子中的 $Char, Bool$; Int, Int 都是类型参数

关于局部类型参数合成

关于局部类型参数合成

- 什么是局部类型参数合成

关于局部类型参数合成

- 什么是<u>局部</u>类型参数<u>合成</u>
  - <u>合成</u>是指我们想将前一页的调用写为 $f(\lambda x : Char \to true, ['a','b','c'])$ 和 $f(\lambda x : Int \to x + 1, [1,2,3])$ 尖括号 $\langle \ldots \rangle$ 中的内容交由编译器计算出

关于局部类型参数合成

- 什么是<u>局部</u>类型参数<u>合成</u>
  - <u>合成</u>是指我们想将前一页的调用写为 $f(\lambda x : Char \to true, ['a','b','c'])$ 和 $f(\lambda x : Int \to x + 1, [1,2,3])$ 尖括号 $\langle \ldots \rangle$ 中的内容交由编译器计算出
  - <u>局部</u>是指

关于局部类型参数合成

- 什么是局部类型参数合成
  - 合成是指我们想将前一页的调用写为 $f(\lambda x : Char \to true, ['a','b','c'])$ 和 $f(\lambda x : Int \to x + 1, [1,2,3])$ 尖括号 $\langle \ldots \rangle$ 中的内容交由编译器计算出
  - 局部是指
    - 函数 f 的类型是事先给定的，不是推出来的

关于局部类型参数合成

- 什么是局部类型参数合成
  - 合成是指我们想将前一页的调用写为 $f(\lambda x : Char \to true, ['a','b','c'])$ 和 $f(\lambda x : Int \to x + 1, [1,2,3])$ 尖括号 $\langle \ldots \rangle$ 中的内容交由编译器计算出
  - 局部是指
    - 函数 f 的类型是事先给定的，不是推出来的
    - 该算法单独考虑函数 f 的每一次调用，不管其他事儿了

关于局部类型参数合成

- 什么是局部类型参数合成
  - 合成是指我们想将前一页的调用写为 $f(\lambda x : Char \to true, ['a','b','c'])$ 和 $f(\lambda x : Int \to x + 1, [1,2,3])$ 尖括号 $\langle \ldots \rangle$ 中的内容交由编译器计算出
  - 局部是指
    - 函数 f 的类型是事先给定的，不是推出来的
    - 该算法单独考虑函数 f 的每一次调用，不管其他事儿了
- 简单定义：即给定函数 f 的类型以及调用 $f(e_1 \ldots e_n)$，补全调用为 $f\langle T_1 \ldots T_n \rangle(e_1 \ldots e_n)$

关于局部类型参数合成

- 什么是局部类型参数合成
  - 合成是指我们想将前一页的调用写为 $f(\lambda x : Char \to true, ['a','b','c'])$ 和 $f(\lambda x : Int \to x + 1, [1,2,3])$ 尖括号 $\langle \ldots \rangle$ 中的内容交由编译器计算出
  - 局部是指
    - 函数 f 的类型是事先给定的，不是推出来的
    - 该算法单独考虑函数 f 的每一次调用，不管其他事儿了
- 简单定义：即给定函数 f 的类型以及调用 $f(e_1 \ldots e_n)$，补全调用为 $f\langle T_1 \ldots T_n \rangle(e_1 \ldots e_n)$
  - 通常还想要 $T_1 \ldots T_n$ 在某些指导思想下是最优的

从简单情形说起

- 类型、子类型关系（记号🈶，细节🈚）、项

$$
\begin{array}{ll}
T ::= & \\
\quad X & \text{类型变元} \\
\mid \forall X <: T . T & \text{多态类型} \\
\mid T * T & \text{元组类型} \\
\mid T \to T & \text{函数类型} \\
\mid Primitive & \text{原始类型} \\
\mid \top & \text{顶类型} \\
\mid \bot & \text{底类型} \\
\mid Unit & \text{单元类型}
\end{array}
$$

子类型关系记号 $T_1 <: T_2$ 表示 T_1 是 T_2 的子类型

$t ::= \ldots$

term，大家都懂的，如 1 + 2, true

- 偷个懒，没在语法上区分多态类型和单态类型

从简单情形说起

从简单情形说起

- 给定 $f : \forall X, Y.\ X * Y \to Y$，如何对 $let\ a : Bool = f((1, true))$执行局部类型参数合成?

从简单情形说起

- 给定 $f : \forall X, Y.\ X * Y \to Y$，如何对 $let\ a : Bool = f((1, true))$执行局部类型参数合成?
  - $1 : Int \quad true : Bool \quad (1, true) : Int * Bool$

从简单情形说起

- 给定 $f : \forall X, Y.\ X * Y \to Y$，如何对 $let\ a : Bool = f((1, true))$执行局部类型参数合成?
  - $1 : Int \quad true : Bool \quad (1, true) : Int * Bool$
  - 函数的<u>实参类型</u>需要是形参类型的子类型，则有 $Int * Bool <: X * Y$

从简单情形说起

- 给定 $f : \forall X, Y . X * Y \to Y$，如何对 $let\ a : Bool = f((1, true))$执行局部类型参数合成?
  - $1 : Int \quad true : Bool \quad (1, true) : Int * Bool$
  - 函数的<u>实参类型</u>需要是形参类型的子类型，则有 $Int * Bool <: X * Y$
  - 函数的<u>返回类型</u>需要是上下文要求的类型的子类型，则有 $Y <: Bool$

从简单情形说起

- 给定 $f : \forall X, Y.\ X * Y \to Y$，如何对 $let\ a : Bool = f((1, true))$执行局部类型参数合成?
  - $1 : Int \quad true : Bool \quad (1, true) : Int * Bool$
  - 函数的<u>实参类型</u>需要是形参类型的子类型，则有 $Int * Bool <: X * Y$
  - 函数的<u>返回类型</u>需要是上下文要求的类型的子类型，则有 $Y <: Bool$
- 于是我们得到包含类型变元 X 和 Y 的<u>子类型"不等式"组</u>

从简单情形说起

- 给定 $f : \forall X, Y\,.\; X * Y \to Y$，如何对 $let\ a : Bool = f((1, true))$执行局部类型参数合成?
  - $1 : Int \quad true : Bool \quad (1, true) : Int * Bool$
  - 函数的<u>实参类型</u>需要是形参类型的子类型，则有 $Int * Bool <: X * Y$
  - 函数的<u>返回类型</u>需要是上下文要求的类型的子类型，则有 $Y <: Bool$
- 于是我们得到包含类型变元 X 和 Y 的<u>子类型"不等式"组</u>
- 最后，解出来就行了，这里能直接看出 X 是 Int，Y 是 $Bool$

从简单情形说起

- 给定 $f : \forall X, Y.\ X * Y \to Y$，如何对 $let\ a : Bool = f((1, true))$执行局部类型参数合成?
  - $1 : Int \quad true : Bool \quad (1, true) : Int * Bool$
  - 函数的<u>实参类型</u>需要是形参类型的子类型，则有 $Int * Bool <: X * Y$
  - 函数的<u>返回类型</u>需要是上下文要求的类型的子类型，则有 $Y <: Bool$
- 于是我们得到包含类型变元 X 和 Y 的<u>子类型"不等式"组</u>
- 最后，解出来就行了，这里能直接看出 X 是 Int，Y 是 $Bool$
- 所以概括说来就是两个步骤：约束生成、约束求解

约束生成

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变
  - 既定的子类型关系，如给定 $S1 <: T1$, $S2 <: T2$，则有 $S1 * S2 <: T1 * T2$

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变
  - 既定的子类型关系，如给定 $S1 <: T1$, $S2 <: T2$，则有 $S1 * S2 <: T1 * T2$
  - 逐步拆解，如将 $Int * Bool <: X * Y$ 根据上述关系拆成 $Int <: X$ 和 $Bool <: Y$

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变
  - 既定的子类型关系，如给定 $S1 <: T1$, $S2 <: T2$，则有 $S1 * S2 <: T1 * T2$
  - 逐步拆解，如将 $Int * Bool <: X * Y$ 根据上述关系拆成 $Int <: X$ 和 $Bool <: Y$
- 可以将此过程称为 unify

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变
  - 既定的子类型关系，如给定 $S1 <: T1$, $S2 <: T2$，则有 $S1 * S2 <: T1 * T2$
  - 逐步拆解，如将 $Int * Bool <: X * Y$ 根据上述关系拆成 $Int <: X$ 和 $Bool <: Y$
- 可以将此过程称为 unify
  - `unify : (Ty, Ty) -> Constraints`，默认第一个 Ty 是第二个 Ty 的子类型

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变
  - 既定的子类型关系，如给定 $S1 <: T1$, $S2 <: T2$，则有 $S1 * S2 <: T1 * T2$
  - 逐步拆解，如将 $Int * Bool <: X * Y$ 根据上述关系拆成 $Int <: X$ 和 $Bool <: Y$
- 可以将此过程称为 unify
  - `unify : (Ty, Ty) -> Constraints`，默认第一个 Ty 是第二个 Ty 的子类型
  - Constraints 即子类型"不等式"的集合

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变
  - 既定的子类型关系，如给定 $S1 <: T1$, $S2 <: T2$，则有 $S1 * S2 <: T1 * T2$
  - 逐步拆解，如将 $Int * Bool <: X * Y$ 根据上述关系拆成 $Int <: X$ 和 $Bool <: Y$
- 可以将此过程称为 unify
  - `unify : (Ty, Ty) -> Constraints`，默认第一个 Ty 是第二个 Ty 的子类型
  - Constraints 即子类型"不等式"的集合
- 对 unify 的参数做 case analysis：

约束生成

- 基本说来，就是根据既定的子类型关系，去按类型的结构逐步拆解，直到找到类型变
  - 既定的子类型关系，如给定 $S1 <: T1$, $S2 <: T2$，则有 $S1 * S2 <: T1 * T2$
  - 逐步拆解，如将 $Int * Bool <: X * Y$ 根据上述关系拆成 $Int <: X$ 和 $Bool <: Y$
- 可以将此过程称为 unify
  - `unify : (Ty, Ty) -> Constraints`，默认第一个 Ty 是第二个 Ty 的子类型
  - Constraints 即子类型"不等式"的集合
- 对 unify 的参数做 case analysis：
  - Case TyVar　　Case Tuple　　Case Func　　…

约束生成

```
unify(arg, param) =

  if arg is ty-var
    return {arg <: param}

  else if param is ty-var
    return {arg <: param}

  else if arg == S1*S2 && param == U1*U2
    return unify(S1,U1) `union` unify(S2,U2)

  else if arg == S1 -> S2 && param == U1 -> U2
    return unify(U1,S1) `union` unify(S2,U2)

  else checkSubtype(arg, param)
```

约束生成

```
unify(arg, param) =

  if arg is ty-var
    return {arg <: param}

  else if param is ty-var
    return {arg <: param}

  else if arg == S1*S2 && param == U1*U2
    return unify(S1,U1) `union` unify(S2,U2)

  else if arg == S1 -> S2 && param == U1 -> U2
    return unify(U1,S1) `union` unify(S2,U2)

  else checkSubtype(arg, param)
```

逆变

约束生成示例

给定 $f : \forall X, Y.\ X * Y \to Y$ 和调用 $let\ a : Bool = f((1, true))$

则参数的类型为 $Int * Bool$，会触发

```
  unify(Int * Bool, X * Y)
= unify(Int, X) `union` unify (Bool, Y)
= {Int <: X, Bool <: Y}
```

要求返回类型是 $Bool$ 的子类型，会触发

```
  unify(Y, Bool)
= {Y <: Bool}
```

所有的约束为 $\{Int <: X, Bool <: Y, Y <: Bool\}$

约束求解

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?
  - 传统方法是根据 X_i 在多态函数定义的返回类型 R 中的型变情况来做决策（请自行学习型变）

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?
  - 传统方法是根据 X_i 在多态函数定义的返回类型 R 中的<u>型变</u>情况来做决策（请自行学习<u>型变</u>）
    - 协变选$\{S_{i1} \ldots S_{in}\}$　　逆变选$\{U_{i1} \ldots U_{in}\}$

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?
  - 传统方法是根据 X_i 在多态函数定义的返回类型 R 中的型变情况来做决策（请自行学习型变）
    - 协变选$\{S_{i1} \ldots S_{in}\}$　　逆变选$\{U_{i1} \ldots U_{in}\}$
  - 但是有些问题

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?
  - 传统方法是根据 X_i 在多态函数定义的返回类型 R 中的<u>型变</u>情况来做决策（请自行学习<u>型变</u>）
    - 协变选$\{S_{i1} \ldots S_{in}\}$　　逆变选$\{U_{i1} \ldots U_{in}\}$
  - 但是有些问题
    - X_i 在 R 中不出现选啥 $\Rightarrow$ 选$\{S_{i1} \ldots S_{in}\}$ $\Rightarrow$ 凭什么?

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?
  - 传统方法是根据 X_i 在多态函数定义的返回类型 R 中的型变情况来做决策（请自行学习型变）
    - 协变选$\{S_{i1} \ldots S_{in}\}$　　逆变选$\{U_{i1} \ldots U_{in}\}$
  - 但是有些问题
    - X_i 在 R 中不出现选啥 $\Rightarrow$ 选$\{S_{i1} \ldots S_{in}\}$ $\Rightarrow$ 凭什么?
    - X_i 在 R 中既协变又逆变选啥 $\Rightarrow$ 选 $\{S_{i1} \ldots S_{in}\}$ $\Rightarrow$ 凭什么?

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?
  - 传统方法是根据 X_i 在多态函数定义的返回类型 R 中的<u>型变</u>情况来做决策（请自行学习<u>型变</u>）
    - 协变选$\{S_{i1} \ldots S_{in}\}$　　逆变选$\{U_{i1} \ldots U_{in}\}$
  - 但是有些问题
    - X_i 在 R 中不出现选啥 $\Rightarrow$ 选$\{S_{i1} \ldots S_{in}\}$ $\Rightarrow$ 凭什么?
    - X_i 在 R 中既协变又逆变选啥 $\Rightarrow$ 选 $\{S_{i1} \ldots S_{in}\}$ $\Rightarrow$ 凭什么?
  - 我们就直接无脑选 $\{S_{i1} \ldots S_{in}\}$了

约束求解

- 由此，得到一组形如 $X <: T$ 或 $T <: X$ 的约束
  - 其中 X 表示要求解的类型变元，T 表示具体类型
- 对每个要求解的类型变元 X_i 合并约束，可得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 那么如何从中得到解呢?
  - 传统方法是根据 X_i 在多态函数定义的返回类型 R 中的型变情况来做决策（请自行学习型变）
    - 协变选$\{S_{i1} \ldots S_{in}\}$　　逆变选$\{U_{i1} \ldots U_{in}\}$
  - 但是有些问题
    - X_i 在 R 中不出现选啥 $\Rightarrow$ 选$\{S_{i1} \ldots S_{in}\}$ $\Rightarrow$ 凭什么?
    - X_i 在 R 中既协变又逆变选啥 $\Rightarrow$ 选 $\{S_{i1} \ldots S_{in}\}$ $\Rightarrow$ 凭什么?
  - 我们就直接无脑选 $\{S_{i1} \ldots S_{in}\}$了
    - 除非它为空（不存在），或者不满足要求（下一页）才选$\{U_{i1} \ldots U_{in}\}$

约束求解

约束求解

- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束

约束求解

- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 求$\{S_{i1} \ldots S_{in}\}$的<u>最小公共父类</u>，如果不为 $\bot$ 或 $\top$，则其为 X_i 的解

约束求解

- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 求$\{S_{i1} \ldots S_{in}\}$的<u>最小公共父类</u>，如果不为 $\bot$ 或 $\top$，则其为 X_i 的解
- 否则 X_i 取$\{U_{i1} \ldots U_{in}\}$的<u>最大公共子类，</u>如果还是为 $\bot$ 或 $\top$，则报错

约束求解

- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 求$\{S_{i1} \ldots S_{in}\}$的<u>最小公共父类</u>，如果不为 $\bot$ 或 $\top$，则其为 X_i 的解
- 否则 X_i 取$\{U_{i1} \ldots U_{in}\}$的<u>最大公共子类，</u>如果还是为 $\bot$ 或 $\top$，则报错
- 图解

约束求解

- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 求$\{S_{i1} \ldots S_{in}\}$的<u>最小公共父类</u>，如果不为 ⊥ 或 ⊤，则其为 X_i 的解
- 否则 X_i 取$\{U_{i1} \ldots U_{in}\}$的<u>最大公共子类，</u>如果还是为 ⊥ 或 ⊤，则报错
- 图解
  - 最小公共父类（*least common supertype* 或 ***l**east **u**pper **b**ound* 或 ***join***) 和

约束求解

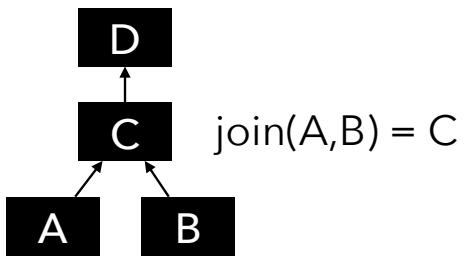
- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 求$\{S_{i1} \ldots S_{in}\}$的<u>最小公共父类</u>，如果不为 ⊥ 或 ⊤，则其为 X_i 的解
- 否则 X_i 取$\{U_{i1} \ldots U_{in}\}$的<u>最大公共子类，</u>如果还是为 ⊥ 或 ⊤，则报错
- 图解
  - 最小公共父类（*least common supertype* 或 ***l**east **u**pper **b**ound* 或 ***join***）和
  - 最大公共子类（*greatest common subtype* 或 ***g**reatest **l**ower **b**ound* 或 ***meet***）

约束求解

- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 求$\{S_{i1} \ldots S_{in}\}$的<u>最小公共父类</u>，如果不为 ⊥ 或 ⊤，则其为 X_i 的解
- 否则 X_i 取$\{U_{i1} \ldots U_{in}\}$的<u>最大公共子类，</u>如果还是为 ⊥ 或 ⊤，则报错
- 图解
  - 最小公共父类（*least common supertype* 或 ***l**east **u**pper **b**ound* 或 ***join***）和
  - 最大公共子类（*greatest common subtype* 或 ***g**reatest **l**ower **b**ound* 或 ***meet***）
  - 细节太多，这次不展开了

约束求解

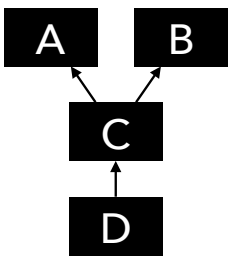
- 得形如 $\{S_{i1} \ldots S_{in}\} <: X_i <: \{U_{i1} \ldots U_{in}\}$ 的约束
- 求$\{S_{i1} \ldots S_{in}\}$的<u>最小公共父类</u>，如果不为 ⊥ 或 ⊤，则其为 X_i 的解
- 否则 X_i 取$\{U_{i1} \ldots U_{in}\}$的<u>最大公共子类，</u>如果还是为 ⊥ 或 ⊤，则报错
- 图解
  - 最小公共父类（*least common supertype* 或 ***l**east **u**pper **b**ound* 或 ***join***）和
  - 最大公共子类（*greatest common subtype* 或 ***g**reatest **l**ower **b**ound* 或 ***meet***）
  - 细节太多，这次不展开了



join(A,B) = C

join(A,B) = ⊤

meet(A,B) = ⊥



meet(A,B) = C

约束求解

- 设求解函数为 solve,
- solve : Constraints -> Map<TyVar, Type>

```
solve(constraints) =
  let sigma = {}
  foreach {S1..Sn} <: X <: {U1..Un} in constraints
    let tyJ = join(S1..Sn)
    if tyJ is not ⊥ or ⊤
      sigma.insert(X -> tyJ)
    else
      let tyM = meet(U1..Un)
      if tyM is not ⊥ or ⊤
        sigma.insert(X -> tyM)
  return sigma
```

约束求解示例

约束求解示例

- 刚才的例子，约束集为 $\{Int <: X, Bool <: Y, Y <: Bool\}$

约束求解示例

- 刚才的例子，约束集为 $\{Int <: X, Bool <: Y, Y <: Bool\}$
- 合并后得约束集 $\{Int <: X, Bool <: Y <: Bool\}$

约束求解示例

- 刚才的例子，约束集为 $\{Int <: X, Bool <: Y, Y <: Bool\}$
- 合并后得约束集 $\{Int <: X, Bool <: Y <: Bool\}$
- X 的下界不为空，其最小公共父类 $join(Int) = Int$，所以得 $X \to Int$

约束求解示例

- 刚才的例子，约束集为 $\{Int <: X, Bool <: Y, Y <: Bool\}$
- 合并后得约束集 $\{Int <: X, Bool <: Y <: Bool\}$
- X 的下界不为空，其最小公共父类 $join(Int) = Int$，所以得 $X \to Int$
- Y 的下界不为空，其最小公共父类 $join(Bool) = Bool$，所以得 $Y \to Bool$

约束求解示例

- 刚才的例子，约束集为 $\{Int <: X, Bool <: Y, Y <: Bool\}$
- 合并后得约束集 $\{Int <: X, Bool <: Y <: Bool\}$
- X 的下界不为空，其最小公共父类 $join(Int) = Int$，所以得 $X \to Int$
- Y 的下界不为空，其最小公共父类 $join(Bool) = Bool$，所以得 $Y \to Bool$
  - 这里<u>求解时</u>其实还要检查下界一定小于上界，或下界的 *join* 一定小于上界的 *meet*

进阶1： 自定义类型

处理自定义类型

处理自定义类型

- 当用户自定义新的类型时

处理自定义类型

- 当用户自定义新的类型时
  - 例如 `class C<X,Y> {}`，`class D<Y>`，且有 `D<Z> <: C<Z,Z>` 这样的继承关系

处理自定义类型

- 当用户自定义新的类型时
  - 例如 `class C<X,Y> {}`，`class D<Y>`，且有 `D<Z> <: C<Z,Z>` 这样的继承关系
- unify 会遇到 arg 和 param 的类型构造器不同的情形

处理自定义类型

- 当用户自定义新的类型时
  - 例如 `class C<X,Y> {}`，`class D<Y>`，且有 `D<Z> <: C<Z,Z>` 这样的继承关系
- unify 会遇到 arg 和 param 的类型构造器不同的情形
  - 例如 `unify(D<X>, C<Int,Int>)`

处理自定义类型

- 当用户自定义新的类型时
  - 例如 `class C<X,Y> {}`，`class D<Y>`，且有 `D<Z> <: C<Z,Z>` 这样的继承关系
- unify 会遇到 arg 和 param 的类型构造器不同的情形
  - 例如 `unify(D<X>, C<Int,Int>)`
- 此时需要将 arg 和 param 的类型构造器变得相同

处理自定义类型

- 当用户自定义新的类型时
  - 例如 `class C<X,Y> {}`，`class D<Y>`，且有 `D<Z> <: C<Z,Z>` 这样的继承关系
- unify 会遇到 arg 和 param 的类型构造器不同的情形
  - 例如 `unify(D<X>, C<Int,Int>)`
- 此时需要将 arg 和 param 的类型构造器变得相同
- 引入函数 $promote : SourceTy \to TargetTyName \to TargetTy$

处理自定义类型

- 当用户自定义新的类型时
  - 例如 `class C<X,Y> {}`，`class D<Y>`，且有 `D<Z> <: C<Z,Z>` 这样的继承关系
- unify 会遇到 arg 和 param 的类型构造器不同的情形
  - 例如 `unify(D<X>, C<Int,Int>)`
- 此时需要将 arg 和 param 的类型构造器变得相同
- 引入函数 $promote : SourceTy \to TargetTyName \to TargetTy$
  -
```
promote(D<Char>, C) = C<Char,Char>
promote(D<X>, C) = C<X,X>
promote(C, D<X>) = error “...”
```

处理自定义类型

```
unify(arg, param) =
  if arg is ty-var
    return {arg <: param}
  else if ...

  else if arg is user-defined-ty and
          param is user-defined-ty
    let arg' = promote(arg, name(param))
    return unify(tyArgs(arg'), tyArgs(param)) `union`
           // 假设类型变元是不变的
           unify(tyArgs(param), tyArgs(arg'))

  else checkSubtype(arg, param)

unify(args, params) = … // perform unify pairwise
```

其中 tyArgs : Ty -> List<Ty>
是抓取类型的类型参数的抓手

如

```
  tyArgs(C<Bool,Bool>)
= [Bool,Bool]
  tyArgs(D<X>)
= [X]
```

约束生成示例

约束生成示例

- 给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$

约束生成示例

- 给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$
- 假设 $[\prime a', \prime b', \prime c']$ 是 $Array\langle Char \rangle$ 类型，并且 $Array\langle Char \rangle <: List\langle Char \rangle$

约束生成示例

- 给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$
- 假设 $['a', 'b', 'c']$ 是 $Array\langle Char \rangle$ 类型，并且 $Array\langle Char \rangle <: List\langle Char \rangle$
- 调用 $f(\lambda x : Char \to true, ['a', 'b', 'c'])$

约束生成示例

- 给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$
- 假设 $['a', 'b', 'c']$ 是 $Array\langle Char \rangle$ 类型，并且 $Array\langle Char \rangle <: List\langle Char \rangle$
- 调用 $f(\lambda x : Char \to true, ['a', 'b', 'c'])$
  - 则参数的类型分别为 $Char \to Bool$ 和 $List\langle Char \rangle$

约束生成示例

- 给定 $f : \forall T_1, T_2 \,.\, ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$
- 假设 $['a', 'b', 'c']$ 是 $Array\langle Char \rangle$ 类型，并且 $Array\langle Char \rangle <: List\langle Char \rangle$
- 调用 $f(\lambda x : Char \to true, ['a', 'b', 'c'])$
  - 则参数的类型分别为 $Char \to Bool$ 和 $List\langle Char \rangle$
- 会触发

```
  unify(Char -> Bool, T1 -> T2)
= unify(T1, Char) `union` unify (Bool, T2)
= {T1 <: Char, Bool <: T2}
```

以及

约束生成示例

- 给定 $f : \forall T_1, T_2 .\ ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$
- 假设 $['a', 'b', 'c']$ 是 $Array\langle Char \rangle$ 类型，并且 $Array\langle Char \rangle <: List\langle Char \rangle$
- 调用 $f(\lambda x : Char \to true, ['a', 'b', 'c'])$
  - 则参数的类型分别为 $Char \to Bool$ 和 $List\langle Char \rangle$
- 会触发

```
  unify(Char -> Bool, T1 -> T2)
= unify(T1, Char) `union` unify (Bool, T2)
= {T1 <: Char, Bool <: T2}
```

以及

```
  unify(Array<Char>, List<T1>
= let List<Char> = promote(Array<Char>, List)
  unify (Char, T1) `union` unify(T1, Char)
= {Char <: T1, T1 <: Char}
```

约束生成示例

- 给定 $f : \forall T_1, T_2 .\ ((T_1 \to T_2), List\langle T_1 \rangle) \to T_2$
- 假设 $['a', 'b', 'c']$ 是 $Array\langle Char \rangle$ 类型，并且 $Array\langle Char \rangle <: List\langle Char \rangle$
- 调用 $f(\lambda x : Char \to true, ['a', 'b', 'c'])$
  - 则参数的类型分别为 $Char \to Bool$ 和 $List\langle Char \rangle$
- 会触发

```
  unify(Char -> Bool, T1 -> T2)
= unify(T1, Char) `union` unify (Bool, T2)
= {T1 <: Char, Bool <: T2}
```

以及

```
  unify(Array<Char>, List<T1>
= let List<Char> = promote(Array<Char>, List)
  unify (Char, T1) `union` unify(T1, Char)
= {Char <: T1, T1 <: Char}
```

- 整理后得 $\{Char <: T1 <: Char, Bool <: T2\}$

进阶2：递归上界

递归上界的情形

递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束

递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y.\ X * Y \to Y$

递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y.\ X * Y \to Y$
- 接下来讨论 f 中的类型变元有<u>上界约束</u>的情形

递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y.\ X * Y \to Y$
- 接下来讨论 f 中的类型变元有<u>上界约束</u>的情形
  - 假设 $I\langle X \rangle$ 是某接口类型，

递归上界的情形

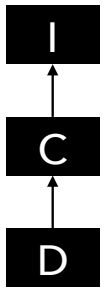
- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y.\ X * Y \to Y$
- 接下来讨论 f 中的类型变元有<u>上界约束</u>的情形
  - 假设 $I\langle X\rangle$ 是某接口类型，
  - C 是 class 类型，$C <: I\langle C\rangle$

递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y.\ X * Y \to Y$
- 接下来讨论 f 中的类型变元有<u>上界约束</u>的情形
  - 假设 $I\langle X\rangle$ 是某接口类型，
  - C 是 class 类型，$C <: I\langle C\rangle$
  - D 是 class 类型，$D <: C$

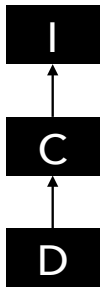
递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y.\ X * Y \to Y$
- 接下来讨论 f 中的类型变元有<u>上界约束</u>的情形
  - 假设 $I\langle X \rangle$ 是某接口类型，
  - C 是 class 类型，$C <: I\langle C \rangle$
  - D 是 class 类型，$D <: C$



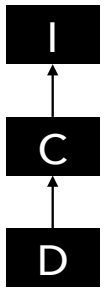
递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y.\ X * Y \to Y$
- 接下来讨论 f 中的类型变元有<u>上界约束</u>的情形
  - 假设 $I\langle X\rangle$ 是某接口类型，
  - C 是 class 类型，$C <: I\langle C\rangle$
  - D 是 class 类型，$D <: C$
  - 给定 $f : \forall Y <: Z, Z <: I\langle Y\rangle.\ Y \to Unit$



递归上界的情形

- 之前讨论的基本情况里，函数定义 f 中的类型变元并无约束
  - 如 $f : \forall X, Y .\ X * Y \to Y$
- 接下来讨论 f 中的类型变元有<u>上界约束</u>的情形
  - 假设 $I\langle X \rangle$ 是某接口类型，
  - C 是 class 类型，$C <: I\langle C \rangle$
  - D 是 class 类型，$D <: C$
  - 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle .\ Y \to Unit$
  - 求解调用 $f(D())$



递归上界的情形

递归上界的情形

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$

递归上界的情形

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$
  - 以及类型定义 $D <: C <: I\langle C \rangle$

递归上界的情形

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$
  - 以及类型定义 $D <: C <: I\langle C \rangle$
  - 求解调用 $f(D())$

递归上界的情形

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$
  - 以及类型定义 $D <: C <: I\langle C \rangle$
  - 求解调用 $f(D())$
- 朴素地想，调用处只能得到 $D <: Y$

递归上界的情形

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$
  - 以及类型定义 $D <: C <: I\langle C \rangle$
  - 求解调用 $f(D())$
- 朴素地想，调用处只能得到 $D <: Y$
  - 整理后也只有 $\{D\} <: Y <: Z; \{\} <: Z <: I\langle Y \rangle$。这可咋整?

递归上界的情形

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$
  - 以及类型定义 $D <: C <: I\langle C \rangle$
  - 求解调用 $f(D())$
- 朴素地想，调用处只能得到 $D <: Y$
  - 整理后也只有 $\{D\} <: Y <: Z; \{\} <: Z <: I\langle Y \rangle$。这可咋整?
  - $Y = D, Z = I\langle Y \rangle$ 然后 $Z = I\langle D \rangle$? $\Longrightarrow$ 显然不满足，因为 $D <: I\langle D \rangle$ 未定义

递归上界的情形

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$
  - 以及类型定义 $D <: C <: I\langle C \rangle$
  - 求解调用 $f(D())$
- 朴素地想，调用处只能得到 $D <: Y$
  - 整理后也只有 $\{D\} <: Y <: Z; \{\} <: Z <: I\langle Y \rangle$。这可咋整?
  - $Y = D, Z = I\langle Y \rangle$ 然后 $Z = I\langle D \rangle$? $\Longrightarrow$ 显然不满足，因为 $D <: I\langle D \rangle$ 未定义
  - 搜索? 太低效!

递归上界的情形——约束生成

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成<u>新约束</u>时，对<u>既存上下界</u>再次<u>递归</u>生成约束

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成新约束时，对既存上下界再次递归生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成<u>新约束</u>时，对<u>既存上下界</u>再次<u>递归</u>生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z
  - 因为 $D <: Y, Y <: Z$，由传递性，必然有 $D <: Z$

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成新约束时，对既存上下界再次递归生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z
  - 因为 $D <: Y, Y <: Z$，由传递性，必然有 $D <: Z$
  - 递归 `unify(D, Z)`

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成新约束时，对既存上下界再次递归生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z
  - 因为 $D <: Y, Y <: Z$，由传递性，必然有 $D <: Z$
  - 递归 `unify(D, Z)`
    - 找到 Z 的上界 $I\langle Y \rangle$，由传递性，必然有 $D <: I\langle Y \rangle$

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成新约束时，对既存上下界再次递归生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z
  - 因为 $D <: Y, Y <: Z$，由传递性，必然有 $D <: Z$
  - 递归 `unify(D, Z)`
    - 找到 Z 的上界 $I\langle Y \rangle$，由传递性，必然有 $D <: I\langle Y \rangle$
    - `unify(D, I<Y>)`

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成新约束时，对既存上下界再次递归生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z
  - 因为 $D <: Y, Y <: Z$，由传递性，必然有 $D <: Z$
  - 递归 `unify(D, Z)`
    - 找到 Z 的上界 $I\langle Y \rangle$，由传递性，必然有 $D <: I\langle Y \rangle$
    - `unify(D, I<Y>)`
      - `I<C> = promote(D, I)`

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成新约束时，对既存上下界再次递归生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z
  - 因为 $D <: Y, Y <: Z$，由传递性，必然有 $D <: Z$
  - 递归 `unify(D, Z)`
    - 找到 Z 的上界 $I\langle Y \rangle$，由传递性，必然有 $D <: I\langle Y \rangle$
    - `unify(D, I<Y>)`
      - `I<C> = promote(D, I)`
      - `unify(C,Y)`，找到 Y 的既存上界 Z，……

递归上界的情形——约束生成

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 以及类型定义 $D <: C <: I\langle C \rangle$
- 方法：每当生成新约束时，对既存上下界再次递归生成约束
  - `unify(D, Y)` 时，找到 Y 的既存上界 Z
  - 因为 $D <: Y, Y <: Z$，由传递性，必然有 $D <: Z$
  - 递归 `unify(D, Z)`
    - 找到 Z 的上界 $I\langle Y \rangle$，由传递性，必然有 $D <: I\langle Y \rangle$
    - `unify(D, I<Y>)`
      - `I<C> = promote(D, I)`
      - `unify(C,Y)`，找到 Y 的既存上界 Z，……
- 总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$

递归上界的情形——约束生成

- 为方便描述算法，将数据结构 Constraints 改为 map，
  $Constraints : TyVar \rightarrow (LowerBounds, UpperBounds)$
- 再将 unify 改成<u>以副作用更新约束集</u>的函数
- 算法可概括为

```
unify(arg, param) =
  if arg is ty-var // X <: T
    upperBounds(arg).insert{param}
    foreach lb in lowerBounds(arg)
      unify(lb, param)

  if param is ty-var // T <: X
    lowerBounds(param).insert{arg}
    foreach ub in upperBounds(param)
      unify(arg, ub)
  ...
```

递归上界的情形——约束生成

- 当然，上面的算法可能会进入无限循环
  - 因为我们没有记录 (arg,param) 有没有被处理过，后续过程可能会再次处理
- 应使用 memoization 记录处理过的 (arg, param) 组合

```
unify(memo, arg, param) =
  let pair = (arg, param)
  if memo `contains` pair
    return
  memo.insert(pair)

  if arg is ty-var
  ...
...
```

递归上界的情形——约束求解

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$
  - 这里解出来 Y 和 Z 都是 join(D, C) = C

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$
  - 这里解出来 Y 和 Z 都是 join(D, C) = C

- 但是啊，可以观察到，Y 的上界是含有另一要求解的类型变元 Z 的，这怎么办?

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$
  - 这里解出来 Y 和 Z 都是 join(D, C) = C

- 但是啊，可以观察到，Y 的上界是含有另一要求解的类型变元 Z 的，这怎么办?
- 例如 $\{Z\} <: Y <: \{\ldots\}$ 和 $\{C\} <: Z <: \{\}$ 怎么求解

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$
  - 这里解出来 Y 和 Z 都是 join(D, C) = C

- 但是啊，可以观察到，Y 的上界是含有另一要求解的类型变元 Z 的，这怎么办?
- 例如 $\{Z\} <: Y <: \{\ldots\}$ 和 $\{C\} <: Z <: \{\}$ 怎么求解
  - 目测可以先求解 Z，再求解 Y，即可

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$
  - 这里解出来 Y 和 Z 都是 join(D, C) = C

- 但是啊，可以观察到，Y 的上界是含有另一要求解的类型变元 Z 的，这怎么办?
- 例如 $\{Z\} <: Y <: \{\ldots\}$ 和 $\{C\} <: Z <: \{\}$ 怎么求解
  - 目测可以先求解 Z，再求解 Y，即可
- 因此需要对所有约束，按要求解的类型变元间的依赖关系，进行拓扑排序

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$
  - 这里解出来 Y 和 Z 都是 join(D, C) = C

- 但是啊，可以观察到，Y 的上界是含有另一要求解的类型变元 Z 的，这怎么办?
- 例如 $\{Z\} <: Y <: \{\ldots\}$ 和 $\{C\} <: Z <: \{\}$ 怎么求解
  - 目测可以先求解 Z，再求解 Y，即可
- 因此需要对所有约束，按要求解的类型变元间的依赖关系，进行拓扑排序
  - 再按拓扑序求解

递归上界的情形——约束求解

- 给定 $f : \forall Y <: Z, Z <: I\langle Y \rangle \,.\, Y \to Unit$，求解调用 f(D())
  - 千辛万苦，总之最后得到了 $\{D, C\} <: Y <: \{Z, C\}$ 和 $\{C, D\} <: Z <: I\{Y\}$
  - 这里解出来 Y 和 Z 都是 join(D, C) = C

- 但是啊，可以观察到，Y 的上界是含有另一要求解的类型变元 Z 的，这怎么办?
- 例如 $\{Z\} <: Y <: \{\ldots\}$ 和 $\{C\} <: Z <: \{\}$ 怎么求解
  - 目测可以先求解 Z，再求解 Y，即可
- 因此需要对所有约束，按要求解的类型变元间的依赖关系，进行拓扑排序
  - 再按拓扑序求解
  - 内容太多，忽略此过程

递归上界的情形——约束求解

递归上界的情形——约束求解

- 有时候拓扑排序也没用，因为类型变元互相依赖，成了一个环，例如之前的例子

递归上界的情形——约束求解

- 有时候拓扑排序也没用，因为类型变元互相依赖，成了一个环，例如之前的例子
  - ……总之最后得到了 $\{D, C\} <: Y <: \{\mathbf{Z}, C\}$ 和 $\{C, D\} <: Z <: I\{\mathbf{Y}\}$

递归上界的情形——约束求解

- 有时候拓扑排序也没用，因为类型变元互相依赖，成了一个环，例如之前的例子
  - ……总之最后得到了 $\{D, C\} <: Y <: \{\mathbf{Z}, C\}$ 和 $\{C, D\} <: Z <: I\{\mathbf{Y}\}$
- 那似乎无法保证一定能解出来，只好尽力而为咯

递归上界的情形——约束求解

- 有时候拓扑排序也没用，因为类型变元互相依赖，成了一个环，例如之前的例子
  - ……总之最后得到了 $\{D, C\} <: Y <: \{\mathbf{Z}, C\}$ 和 $\{C, D\} <: Z <: I\{\mathbf{Y}\}$
- 那似乎无法保证一定能解出来，只好尽力而为咯

- 更新 join 和 meet 函数，让其在计算时忽略正在求解的类型变元,，以 meet 为例

递归上界的情形——约束求解

- 有时候拓扑排序也没用，因为类型变元互相依赖，成了一个环，例如之前的例子
  - ……总之最后得到了 $\{D, C\} <: Y <: \{\mathbf{Z}, C\}$ 和 $\{C, D\} <: Z <: I\{\mathbf{Y}\}$
- 那似乎无法保证一定能解出来，只好尽力而为咯

- 更新 join 和 meet 函数，让其在计算时忽略正在求解的类型变元,，以 meet 为例
  - 以前 `meet : (Ty , Ty) -> Ty`

递归上界的情形——约束求解

- 有时候拓扑排序也没用，因为类型变元互相依赖，成了一个环，例如之前的例子
  - ……总之最后得到了 $\{D, C\} <: Y <: \{\mathbf{Z}, C\}$ 和 $\{C, D\} <: Z <: I\{\mathbf{Y}\}$
- 那似乎无法保证一定能解出来，只好尽力而为咯

- 更新 join 和 meet 函数，让其在计算时忽略正在求解的类型变元,，以 meet 为例
  - 以前 `meet : (Ty , Ty) -> Ty`
  - 现在 `meet : ([Ty] , Ty, Ty) -> Ty`

递归上界的情形——约束求解

- 有时候拓扑排序也没用，因为类型变元互相依赖，成了一个环，例如之前的例子
  - ……总之最后得到了 $\{D, C\} <: Y <: \{\mathbf{Z}, C\}$ 和 $\{C, D\} <: Z <: I\{\mathbf{Y}\}$
- 那似乎无法保证一定能解出来，只好尽力而为咯

- 更新 join 和 meet 函数，让其在计算时忽略正在求解的类型变元,，以 meet 为例
  - 以前 `meet : (Ty , Ty) -> Ty`
  - 现在 `meet : ([Ty] , Ty, Ty) -> Ty`

```
meet (ignored, ty1, ty2) =
  if ignored `contains` ty1
    return ty2
  if ignored `contains` ty2
    return ty1
  … // the same old code
```

进阶3：上界有和类型，下界有或类型

上界有交类型的情形

- 反正也讲不完。下次吧……